

GEAR-RCA: A Graph-Guided Episodic Recall Agent for Fintech-Infrastructure Root-Cause Analysis

A Controlled Ablation on a Live-Injected Benchmark

ClearFlow-RCA Project
Real-Time Payment Cascade Intelligence
ClearFlow-RCA
Draft, September 2026

Abstract—We build GEAR-RCA, a Graph-Guided Episodic Recall Agent for Fintech-Infrastructure Root-Cause Analysis: a system for a live eight-service payments platform, assembled from three components that prior work has only combined in pairs: a real code-dependency graph (1,294 nodes, built by static AST parsing over the actual service codebase, not a synthetic topology) that the agent queries as a tool rather than a fixed ranking computed once per incident; a tool-calling large language model (LLM) agent that drives a multi-round investigation loop over that graph plus live logs, traces, and infrastructure health, with structural enforcement replacing prompted-but-ignored instructions; and a three-tier episodic memory (a curated known-issues note, leave-one-out case recall over past confirmed incidents, and per-fault-type strategy distillation extracted from both successful and failed past investigations), layered on top of the graph-guided LLM loop rather than replacing it. All 97 confirmed incidents used for scoring come from faults triggered against a live, running system rather than replayed or synthetically generated telemetry; to our knowledge this is the first RCA benchmark in this literature built this way. On the 11 incidents where every prior method scored 0% accuracy-at-1 (AC@1) at the outset of this project, the memory-equipped agent reaches 81.8% (fair-credit AC@1), against 72.7% for an otherwise identical no-memory reimplement of a recent graph-guided competitor’s described methodology, a controlled ablation isolating a +9.1 point effect. Across a full 97-case confirmed benchmark built from real fault injections against the live system, the memory-equipped agent reaches 78.4% literal / 83.5% fair-credit AC@1. The one fault type for which no strategy could be distilled (insufficient case count) scores 0%, the single clearest piece of evidence that the learned-strategy tier, and not the tool set alone, drives the improvement we attribute to memory. We report two real data-leakage bugs found and fixed mid-study, including one whose diagnosis was triggered by a collaborator’s question about how memory storage worked, and we treat that disclosure as part of the method rather than an incidental footnote.

Index Terms—root cause analysis, agent memory, large language model agents, tool use, AIOps, episodic memory, case-based reasoning

I. INTRODUCTION

Root-cause analysis (RCA) in microservice systems has moved from purely statistical and graph-theoretic methods

toward large language model (LLM) agents that reason over logs, traces, and metrics using tool calls. A parallel line of work has begun equipping such agents with persistent memory across incidents, on the premise that a system which has diagnosed a fault before should not have to rediscover it from scratch. Both directions are active in 2026, separately: graph-guided LLM investigation without memory [8], and memory-augmented LLM investigation without an explicit code-dependency graph [6], [7].

This paper introduces GEAR-RCA, a system that combines both, applied to a domain the closest prior work does not address: payment processing, where a root cause is frequently not an infrastructure failure but a *business-domain state fact*, such as an anti-money-laundering (AML) sanctions hold or an idempotency-key collision, that no purely infrastructure-telemetry method is built to recall. We built the system incrementally, testing each component in isolation before combining them, and we report a controlled ablation (memory enabled vs. disabled, all other components held fixed) rather than a single end-to-end number, so that the paper’s central claim is falsifiable and precise rather than an aggregate improvement of unclear origin.

Concretely, the system we built and evaluate has three layers, each doing a distinct job rather than duplicating the others:

- **Graph.** A real code-dependency graph over the eight-service codebase (1,294 nodes), extracted by static AST parsing plus semantic analysis of the actual source, not a hand-drawn or synthetic service topology. The agent queries it as a live tool mid-investigation (`query_knowledge_graph`) to ground a candidate root cause in an actual code dependency, rather than consulting a ranking precomputed once and fixed for the incident.
- **LLM agent.** A multi-round (up to 15) tool-calling loop over `gpt-oss-20b`, driving seven tools: the graph query above, live Elasticsearch search over full (not pre-

filtered) log data, a live service-dependency endpoint, real container-health checks, cross-service payment-trace reconstruction, a fallible rule-based funnel-stall heuristic, and the memory-recall tool below. Structural enforcement (Section III-B) makes sure a “mandatory” tool call cannot be skipped under budget pressure, the failure mode we found prompting alone does not prevent.

- **Memory.** A three-tier episodic memory (Section III) layered on top of, not instead of, the graph-guided LLM loop: a static known-issues note, leave-one-out full-text case recall over past confirmed incidents, and per-fault-type strategy distillation that turns accumulated successes *and* failures into concrete guidance injected into the system prompt.

The central empirical question this paper answers is what the memory layer adds once the graph and the tool-using LLM are already in place and held fixed. It is not whether an LLM agent can do RCA at all, and not whether a graph helps a memory-free agent; both questions are addressed separately by prior work (Section II).

Our contributions are as follows. First, a working, tool-using RCA agent evaluated against 97 confirmed incidents from real fault injections on a live 8-service payments stack, not synthetic telemetry replay. To our knowledge, this is the first RCA benchmark in this literature built entirely from faults triggered against a system that was actually running (real process kills, real container stops, real duplicate submissions, real SDN-matching payments) rather than replayed, pre-recorded, or synthetically generated telemetry, with ground truth taken directly from the trigger mechanism itself and confirmed by an independent concurrently-running health-witness process rather than asserted by the benchmark author. Second, a three-tier episodic memory system, with the strategy-distillation tier following the title/description/content schema of ReasoningBank [2], applied for the first time (to our knowledge) to business-domain RCA rather than web-agent or software-engineering tasks. Third, a controlled ablation isolating memory’s real, measured, and non-uniform contribution. Fourth, two disclosed methodological findings: a memory-persistence bug that silently dropped 27 of 28 completed investigations from the case store, and a memory leave-one-out leakage bug, both reported as part of the method rather than omitted.

II. RELATED WORK

A. Agent memory: a three-stage taxonomy

A 2026 ACL Findings survey [1] organizes LLM agent memory into three evolutionary stages: *storage* (raw trajectory preservation, retrieved by similarity), *reflection* (trajectories refined, summarized, or critiqued before retention), and *experience* (trajectories abstracted into reusable strategies, generalized across trajectories). Six memory *types* cut across these stages: in-context working memory, external episodic memory, retrieval-based semantic memory, reflective memory, experience/reasoning memory, and parametric (neural, test-time-updated) memory [5]. Our system spans three of

these: external episodic memory (Section III, Tier 2), experience/reasoning memory (Tier 3), and the in-context working memory every tool-using agent has by construction. Tier 2’s full-text case recall follows the retrieve-and-reuse pattern that case-based reasoning surveys of LLM agents [11] identify as the dominant design for episodic memory in this space, and our choice to build episodic memory at all follows directly from the position that it is a missing, not optional, piece of long-term LLM agents [3]. We do not implement reflective memory (prospective/retrospective refinement in the style of [4]) or parametric memory, and we discuss both as open work in Section VIII.

B. Closest prior systems

Table I summarizes the closest 2026 systems found through a literature search covering the full intersection of agent memory, tool-using agents, and RCA/AIOps. Every close competitor is infrastructure- or telemetry-scoped; none targets a root cause that is a business-domain fact rather than a service failure. GALA+ [8], the closest graph-guided (memory-free) system, and AMER-RCL [6] / OpsMem [7], the closest memory-augmented (largely graph-free) systems, are architecturally complementary to our own design but none combine graph-guided exploration, episodic memory, and structural tool-call enforcement in one system. Procedural-compliance work [12], [13] independently identifies and formally addresses the exact failure mode we encountered and hand-patched (Section III-B): an LLM agent ignoring a “mandatory” instruction under tool-call budget pressure. None of that work has a memory dimension.

C. RCA outside the LLM-agent literature

A separate, older line of RCA work never involves an LLM agent at all, and is post-failure and non-memory by construction. CIRCA [15] treats root-cause analysis as intervention recognition over a causal Bayesian network and improves top-1 recall, but assumes a predefined causal graph rather than querying one live, and produces machine-readable indicators rather than an investigation an operator can follow. CCLH [16] models inter-service relationships as heterogeneous hypergraphs and performs localization through cascaded conditional learning; like CIRCA, it has no cross-incident memory and stops once the current anomaly is explained. Lin *et al.* [17] apply frequent-itemset mining (Apriori, FP-Growth) over structured logs at Facebook’s production scale, demonstrating that classical, non-learned methods still localize root causes across millions of entities. AnoMod [18] contributes a multimodal anomaly-detection-and-RCA dataset for microservices and explicitly names the same gap this paper targets: none of these methods addresses safe introspection or auditability for a regulated domain. All four are complementary evidence that RCA-as-localization is well studied; none of them asks what happens when the same agent investigates the same kind of fault twice, which is the question this paper answers.

Two further papers motivate treating payments as its own domain rather than generic infrastructure. Narayanan [19] catalogs failure modes specific to distributed payment platforms (authentication gaps, race conditions, idempotency violations, multi-rail inconsistencies) and pairs each with an engineering mitigation, but the taxonomy is descriptive: it offers no automated diagnosis and no way to learn from a past incident. Porcelli [20] models inter-message processing time in a real ISO 20022 settlement system (TIPS) as an anomaly-detection feature, one of the few studies that maps directly onto a message-based payment pipeline the way our benchmark does, but stops at localization with no causal reconstruction. Both papers support this paper’s premise (Section IV-A) that a payment root cause is often a domain-specific state fact rather than a generic infrastructure symptom, without either paper building the agent that recalls it.

III. SYSTEM

GEAR-RCA is the three layers introduced in Section I: a real code-dependency graph queried live rather than precomputed, an LLM tool-calling loop that drives the investigation, and a three-tier memory layered on top. This section describes each in the order the agent actually uses them: the LLM loop and its tools (including the graph) first, then memory.

The agent is a multi-round tool-calling loop (up to 15 rounds) driving `gpt-oss-20b` via the NVIDIA NIM API, with seven tools: a rule-based funnel-stall heuristic (called first, explicitly labeled fallible); a live Elasticsearch query tool returning full log data rather than a pre-filtered sample; a real code-dependency graph query (`query_knowledge_graph`, 1,294 nodes, built via static AST parsing plus semantic extraction over the actual service codebase); a live service-dependency endpoint; a container-health check (real `docker inspect`); a cross-service payment-trace reconstruction tool; and the memory-recall tool described below. The graph tool is queried mid-investigation against the specific services implicated by the evidence gathered so far, not consulted once as a precomputed ranking.

A. Three-tier memory

Tier 1 (known issues). A small curated list of confirmed, recurring, unrelated noise sources (one entry: a persistent background error on the settlement service that twice misled independent investigations before being documented). Static, not learned.

Tier 2 (episodic case recall). A SQLite database with full-text search (FTS5) over every past case’s real evidence text. When the agent finds a distinctive log signature, it queries this store and receives matching past cases with their *confirmed* root causes. Queries exclude the incident currently under test (leave-one-out), described further in Section IV-D.

Tier 3 (distilled strategy). Following ReasoningBank’s schema [2], we distill each fault type’s accumulated cases, both successful and failed past investigations, explicitly labeled as such, into one reusable strategy (title, one-sentence description, and 3–5 sentences of concrete guidance), injected directly

into the system prompt rather than offered as a callable tool. Coverage: 11 of 13 fault types; two (insufficient case count) have none, and Section V shows the resulting accuracy gap is stark.

B. Structural enforcement over prompted compliance

An instruction marked “MANDATORY” in the system prompt was routinely ignored once the model was under tool-call budget pressure: it would exhaust its call budget on log search and answer without ever invoking the graph-query or memory-recall tools. We replaced the soft instruction with structural enforcement: the loop tracks whether each required tool has actually been called, and if the model attempts to answer (or exhausts its round budget) without having called it, the next call is forced via the API’s own `tool_choice` parameter rather than repeated as text.

IV. BENCHMARK AND METHODOLOGY

A. Premise: why a live-injected benchmark

We start from a first-principles objection to how this literature evaluates RCA agents. An agent that scores well against replayed or pre-recorded telemetry has been shown to pattern-match a fixed, already-known record of what happened. It has not been shown to investigate a system that is still changing while it looks. The two are not the same skill, and the benchmarks in this space do not distinguish them: they replay archived telemetry, or inject faults into a small demo application built to be broken (Table I), not into a system with the scale, real datastore dependencies, and real business-domain state that a production payments platform actually has. This is not a fringe practice, either: even a 2025 Microsoft Research causal-RCA method [21] is evaluated on PetShop [22], a widely used benchmark of purely synthetic latency and availability time series with injected performance issues, no logs, no source code, and no business-domain state at all. If the state of the art at a major industrial research lab is still validated this way, the gap this paper targets is real, not a straw man. If the claim is that an agent can do root-cause analysis, the evidence should come from causes it actually had to uncover, in a system large and real enough that the fault could plausibly hide the way it would in practice.

That objection is why we built the platform before we built the benchmark. We stood up a real eight-service payments infrastructure (Spring Boot, Kafka, Elasticsearch, MongoDB, Cassandra, Redis) processing real traffic end to end: fraud scoring, validation, AML screening, routing, settlement, audit, rather than a diagram or a toy app, specifically so that a fault injected into it would have somewhere real to propagate and something real to break. Only once that system was live and taking steady traffic did we inject faults into it and record what happened, rather than deciding in advance what an incident would look like and writing telemetry to match. This section describes that benchmark: how faults were triggered against the live system (§IV-B), how a prediction is scored against it (§IV-C), and a leakage bug we found and fixed in the memory system’s interaction with it (§IV-D).

TABLE I
CLOSEST PRIOR SYSTEMS (2026 UNLESS NOTED)

System	Memory design	Domain	What it does not cover
AMER-RCL [6]	Temporal-window cross-alert reasoning reuse, recursive per-alert refinement	Generic microservices	Infra telemetry only; no business-state signal; no public code
OpsMem [7]	Dual short/long-term memory, “cross-memory resonance,” LTM consolidated post-resolution	Huawei production microservices	Infra-only; proprietary dataset, no code
GALA+ [8]	None—bounded graph exploration (STRIX ranking) fresh per incident	OnlineBoutique / TrainTicket / AegisLab, 7 LLMs	Zero cross-incident memory by design; not directly comparable to our numbers (Section VI)
KRCA [9]	Production agentic pipeline (Kuaishou)	Hyper-scale infra	Industry report, not reproducible
OM-RAG [10]	Symptom—root-cause—resolution triples, embedding retrieval	Software bugs (GitHub issues)	Not infrastructure incidents; zero business-state signal by design
AgentLTL [12]	N/A (procedural compliance, not memory)	General tool use	Solves our enforcement problem but has no memory dimension
CIRCA [15]	None—causal Bayesian network, pre-defined per deployment	Generic microservices	No LLM agent, no memory, requires a predefined causal graph
CCLH [16]	None—heterogeneous hypergraph, fresh per incident	Generic microservices	No LLM agent, no memory, post-failure only
AnoMod [18]	N/A (dataset, not a system)	Generic microservices	RCA benchmark with no agent, no memory, no business-state signal

B. Live fault injection, not synthetic replay

Every close prior benchmark in this literature is built from either synthetic telemetry replay or fault injection against a demo/toy application (Table I): GALA+ injects into OnlineBoutique, TrainTicket, and AegisLab [8]; RCAEval likewise targets demo microservice apps with pre-recorded telemetry [14]; OM-RAG scores against archived GitHub issue histories rather than live incidents [10]. To our knowledge, no prior published RCA-agent benchmark is built from faults triggered against a real, running, non-demo system with ground truth taken from the trigger mechanism itself rather than from a pre-existing incident record or an author-asserted label. We built one.

We built 110 gold cases from real faults injected against a live, running 8-service payments platform (Spring Boot, Kafka, Elasticsearch, MongoDB, Cassandra, Redis) via five distinct real trigger mechanisms: real process kill/restart (docker-orchestrated crash of the actual Spring Boot process, not a simulated failure flag), real container stop/start on shared infrastructure dependencies (Redis, MongoDB, Cassandra), a real concurrent decoy-traffic burst that gets genuine HTTP-level rejections from a non-root-cause service (engineering an authentic confound rather than asserting one), real duplicate payment submissions that trigger the gateway’s actual idempotency logic, and real payments built from sanctions-list-matching entities that must be confirmed HELD by querying the live compliance-holds endpoint before being accepted as a labeled case. Ground truth for crash-based faults is tautological (the root service is the exact process the injector killed); for the two payment-domain fault types, a *confirmed* flag records whether the intended effect (a real HELD status, a real 409 duplicate rejection) actually manifested, since those two can legitimately fail to land on a given attempt. Every incident also has an independent health-witness process polling service

health concurrently with, but with no foreknowledge of, the injected fault, and a 240-second cooldown between incidents to prevent one incident’s crash effects from contaminating the next incident’s evidence baseline. Of the 110 cases, 97 are *confirmed* (independently verifiable evidence exists tying the fault to its root cause) and are used for accuracy scoring; the remaining 13 are evidence-free by design (e.g., faults with no discoverable symptom under the current telemetry) and are retained to test correct abstention rather than accuracy.

C. Fair-credit scoring

A prediction of the consuming service (audit or settlement) is credited as correct when the gold label is the underlying datastore (*cassandra*), because both are the *only* two services with a real dependency on Cassandra (verified directly from build manifests, not assumed), and a model naming the affected service is diagnosing the real failure propagation correctly, merely not naming the datastore itself. This rule is not extended to other datastores (e.g. MongoDB) without the equivalent verification, and we report literal and fair-credit accuracy separately throughout.

D. Leave-one-out memory exclusion

Because every gold case is also seeded into the episodic memory store, a naive implementation of `recall_similar_past_incidents` would let the agent retrieve its own answer key during evaluation. We verified this leakage occurred prior to a fix (one case returned as its own top match) and applied an explicit exclusion of the current incident’s own identifier from every recall query, then reverified the fix.

TABLE II
FAIR-CREDIT AC@1 BY FAULT TYPE, FULL 97-CASE BENCHMARK,
MEMORY ENABLED

Fault type	n	Fair-credit AC@1
DB_TIMEOUT	9	100%
KAFKA_CONSUMER_LAG	9	100%
NETWORK_LATENCY	9	100%
AML_HOLD	9	89%
CPU_SATURATION	9	89%
SETTLEMENT_DB_FAILURE_	8	88%
KAFKA_CONFOUND		
SETTLEMENT_DB_FAILURE_	8	88%
LIQUIDITY_CASCADE		
VALIDATION_SLOWDOWN_	9	78%
GATEWAY_CONFOUND		
AML_SERVICE_DEGRADATION_	8	75%
RETRY_CASCADE		
MONGODB_OUTAGE	8	75%
CASSANDRA_OUTAGE	8	62%
REDIS_OUTAGE (no strategy)	3	0%

TABLE III
MEMORY ON/OFF ABLATION, SAME TOOLS OTHERWISE

Condition	n	Literal	Fair-credit
Memory ON, 11 hardest cases	11	45.5%	81.8%
Memory OFF, same 11 cases	11	27.3%	72.7%
Memory ON, full benchmark	97	78.4%	83.5%
Memory OFF, full benchmark	97	66.0%	74.2%

V. EXPERIMENTS AND RESULTS

A. Full benchmark, memory enabled

Across all 97 confirmed cases, the memory-equipped agent reaches 78.4% literal AC@1 (76/97) and 83.5% fair-credit AC@1 (81/97). Table II breaks this down by fault type. Three fault types reach 100%; the two fault types this project has historically found hardest, AML_HOLD and CASSANDRA_OUTAGE, reach 89% and 62% respectively, up from 0% for every method tried before this system existed. The sole fault type with no distilled strategy, REDIS_OUTAGE (three confirmed cases, below the threshold we judged safe to generalize from), scores 0%. That is the clearest single result in this study that the strategy-distillation tier, not the tool set alone, is responsible for the gains on repeated-pattern fault types.

B. Controlled ablation: memory on vs. off

We isolate memory’s contribution by flipping a single flag that disables all three memory tiers and removes the recall tool from the tool list entirely, leaving graph-guided exploration, live telemetry, and trace reconstruction with no cross-incident memory. This configuration is architecturally the closest match to GALA+’s described methodology [8], reimplemented from the published description since GALA+’s code is not public. Table III reports the result on the 11 hardest cases (the subset on which every prior method scored 0% before this system existed) and on a full-benchmark checkpoint.

On the 11 hardest cases, memory contributes a clean +9.1-point effect. A case-level breakdown shows this is real but

noisy: of 11 cases, 6 have identical outcomes with memory on or off, memory wins 3, and the no-memory condition wins 2, including one CASSANDRA_OUTAGE case where the memory-enabled agent guessed a service with no real Cassandra dependency, a genuine confusion introduced by memory rather than a neutral miss. We report this mixed pattern rather than only the aggregate, since a blanket “memory improves RCA” claim does not survive case-level inspection. The more precise claim, that memory helps disproportionately on repeated-pattern fault types with real variance case to case, does.

At full-benchmark scale the same direction holds and the gap widens: memory contributes +12.4 points literal (66.0%→78.4%) and +9.3 points fair-credit (74.2%→83.5%) across all 97 confirmed cases, with identical tools, graph, and enforcement in both arms. The only difference is whether the three memory tiers and the recall tool are present. The no-memory arm’s specific failures are informative: every MONGODB_OUTAGE case (3/3, all predicted gateway or settlement) and every REDIS_OUTAGE case (3/3, all predicted settlement) missed both literal and fair credit without memory, two fault types for which no other datastore-equivalence rule applies, so neither can be rescued by the fair-credit convention the way Cassandra cases can. Both are fault types Section V already identifies as lacking either a distilled strategy (REDIS_OUTAGE) or, in the memory-off condition, any accumulated case history at all, consistent with the hardest-cases finding that the strategy-distillation tier, not the tool set alone, is what memory contributes. By contrast, all three CASSANDRA_OUTAGE misses in the no-memory arm still earned fair credit (the agent correctly named settlement, a real Cassandra-dependent consumer, without literally naming cassandra), the same audit/settlement equivalence described in Section V.

C. Attribution: memory vs. earlier fixes

A direct-signal rule (search for a fault type’s known decisive log line, e.g. an AML sanctions-hit event, before trusting funnel-stall inference) was added before the memory system existed and is gated only by fault type, not by the memory-enabled flag, so it is present identically in both arms of the ablation. In isolated testing, this rule alone corrected 4 of 5 AML_HOLD misses. Because the flag is wired this way, the ablation’s measured gap is attributable specifically to the memory tiers built afterward, not to this earlier, separable fix.

VI. ON EXTERNAL COMPARISON

GALA+ reports 74.44% AC@1 on OnlineBoutique and 73.33% on TrainTicket, both synthetic infrastructure-fault benchmarks on demo e-commerce applications [8]. Our 83.5% fair-credit figure is *not* a valid “exceeds GALA+” claim: different benchmark, different fault domain (business-state payment faults vs. generic infrastructure faults), and a different, explicitly disclosed scoring convention. The only valid comparison is the in-benchmark ablation of Table III. We note two methodological practices GALA+ follows that we

do not: averaging every result over three runs to control for LLM sampling non-determinism (we report single-run results per case), and using human evaluators to score evidence groundedness as a distinct dimension, which surfaced a real weakness common to every method GALA+ tested [8]. We also note a striking convergent finding: GALA+’s own paper discloses that its BARO baseline systematically overpredicts a Redis-backed service because of persistent, unrelated disk-I/O spikes. Independently, our own weakest fault type is also Redis-related (REDIS_OUTAGE, Section V), via an unrelated mechanism (missing memory strategy rather than spurious metric dominance). That suggests Redis-adjacent telemetry is a broadly hard signal-to-noise problem in this literature, not a defect specific to either system.

VII. DISCLOSED BUGS, REPORTED AS METHOD

We report two data-integrity issues found and fixed during this study, both directly affecting the memory system’s validity, rather than omitting them. First, the case-memory store’s uniqueness constraint originally keyed on incident identifier alone; because every gold case is pre-seeded, an agent’s own investigation of a case already in memory silently failed to persist. Before this was found (triggered by a collaborator asking how memory storage worked, not by an automated check), only one of 28 completed agentic investigations had actually been recorded. We fixed this with a composite (incident identifier, source) key and migrated the live database without data loss; all accuracy numbers in this paper are unaffected, since predictions come from live tool calls against real evidence rather than from memory lookup, but one earlier causal claim about memory compounding within a single run was retracted once the bug was found. Second, the leave-one-out exclusion described in Section IV-D was itself a bug caught and fixed before being reported as a result.

VIII. LIMITATIONS

We do not average over multiple runs to control for LLM sampling non-determinism, unlike GALA+. The fair-credit scoring rule, while grounded in verified service-dependency data, has not been checked against an independent or human judgment. One fault type (IDEMPOTENCY_COLLISION_STORM) currently has zero usable cases in memory, the same class of gap as REDIS_OUTAGE, not yet investigated. Reflective memory (Tier 2.5, prospective/retrospective consolidation in the style of [4]) remains unbuilt; the infrastructure to log every investigation’s outcome exists and would support it, but nothing currently consumes that log in a reflective way.

IX. CONCLUSION

We present GEAR-RCA, a tool-using RCA agent with a three-tier episodic memory, evaluated on a live-injected 97-case payments benchmark, and isolate memory’s contribution via a controlled ablation rather than reporting a single aggregate number. The defensible contribution is narrower than

“memory improves RCA”: episodic memory indexed specifically over business-domain state, combined with structural tool-call enforcement, measured with a disclosed leave-one-out protocol and two found-and-fixed leakage bugs, on fault types (AML holds, idempotency collisions) that no close 2026 competitor’s architecture targets. The ablation shows this contribution is real but concentrated on repeated-pattern fault types rather than a uniform lift, a precise claim we prefer over an inflated one.

REFERENCES

- [1] J. Luo, Y. Tian, C. Cao, Z. Luo, H. Lin, K. Li, C. Kong, R. Yang, and J. Ma, “From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms,” *Findings of ACL 2026*, arXiv:2605.06716.
- [2] Google Research, “ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory,” arXiv:2509.25140, 2026.
- [3] M. Pink, Q. Wu, V. A. Vo, J. Turek, J. Mu, A. Huth, and M. Toneva, “Position: Episodic Memory is the Missing Piece for Long-Term LLM Agents,” arXiv:2502.06975, 2025.
- [4] “In Prospect and Retrospect: Reflective Memory Management for Long-term Personalized Dialogue Agents,” *ACL*, 2025, arXiv:2503.08026.
- [5] Google Research, “Titans + MIRAS: Helping AI have long-term memory,” 2025–2026.
- [6] “Agentic Memory Enhanced Recursive Reasoning for Root Cause Localization in Microservices,” arXiv:2601.02732, 2026.
- [7] Y. Sun, R. Gao, Y. Luo, W. Gu, S. Zhang, Q. Guo, Q. Fu, Y. Wu, and D. Pei, “OpsMem: Dual-Memory Reasoning with Cross-Memory Resonance for Failure Diagnosis,” *IEEE ISSRE*, 2026, arXiv:2607.11357.
- [8] “GALA: Graph-Augmented LLM Agents for Root Cause Analysis and Incident Response in Microservices,” arXiv:2608.08968, 2026.
- [9] “KRCA: An Efficient Root Cause Analysis System in Hyper-scale Microservice Systems via Agentic AI,” arXiv:2607.01788, 2026.
- [10] “Leveraging Resolved Incident History for LLM-Assisted Software Bug Diagnosis,” arXiv:2607.21911, 2026.
- [11] “Review of Case-Based Reasoning for LLM Agents: Theoretical Foundations, Architectural Components, and Cognitive Integration,” arXiv:2504.06943, 2025.
- [12] “AgentLTL: A Trace-Verification Framework for Measuring, Enforcing, and Training Procedural Compliance in Tool-Using LLM Agents,” arXiv:2607.02599, 2026.
- [13] “Verified Tool Calls Improve LLM Agent Reliability Under Non-Atomic Failures,” arXiv:2608.02645, 2026.
- [14] L. Pham et al., “RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data,” *FSE*, 2026 (also *WWW* 2025, *ASE* 2024).
- [15] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, “Causal Inference-Based Root Cause Analysis for Online Service Systems with Intervention Recognition,” in *Proc. 28th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD)*, 2022.
- [16] S. Xie, H. He, J. Wang, and B. Li, “Root Cause Analysis for Microservice Systems via Cascaded Conditional Learning with Hypergraphs,” *arXiv preprint*, 2025.
- [17] F. Lin, K. Muzumdar, N. P. Laptev, M.-V. Culelea, S. Lee, and S. Sankar, “Fast Dimensional Analysis for Root Cause Investigation in a Large-Scale Service Environment,” *Proc. ACM on Measurement and Analysis of Computing Systems (SIGMETRICS)*, vol. 4, no. 2, 2020.
- [18] K. Ping, H. B. Mazhar, Y. Wang, Y. Song, and M. V. Mäntylä, “AnoMod: A Dataset for Anomaly Detection and Root Cause Analysis in Microservice Systems,” in *Proc. 23rd Int. Conf. on Mining Software Repositories (MSR)*, 2026.
- [19] V. Narayanan, “The Hidden Failure Modes in Digital Payment Platforms and How to Engineer Around Them,” *Journal of Information Systems Engineering and Management*, 2025.
- [20] L. Porcelli, “A Feature Engineering Approach for Business Impact-Oriented Failure Detection in Distributed Instant Payment Systems,” *arXiv preprint*, 2025.
- [21] L. Nagalapatti, A. Srivastava, S. Sarawagi, and A. Sharma, “Robust Root Cause Diagnosis using In-Distribution Interventions,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025.

- [22] M. Hardt, W. R. Orchard, P. Blöbaum, S. Kasiviswanathan, and E. Kirschbaum, “The PetShop Dataset: Finding Causes of Performance Issues across Microservices,” *Proc. 3rd Conf. on Causal Learning and Reasoning (CLear)*, PMLR, vol. 236, 2024.